

Enhanced Levy Flight Mechanism (LFM) on Binary Dragonfly Algorithm

M. Sai Kumar^{#1} Dr. S. N. Tirumala Rao^{#2} P. John Subhani^{#3}

K. S. V. S. Chandra Prasad^{#4} N. Rajesh^{#5}

**#Computer Science Department, Narasaraopeta Engineering College
Narasaraopeta, Andhra Pradesh, India.**

1 saikumarmamidala1358@gmail.com

2 nagatirumalarao@gmail.com

3 johnsubhanipatan@gmail.com

4 kattamurisiva.2001@gmail.com

5 rajeshnelaturi111@gmail.com

ABSTRACT

Feature selection (FS) is considered as one of the core concepts in the areas of machine learning and data mining which immensely impacts the performance of classification model. Through FS, irrelevant or partially relevant features can be eliminated which in turn helps in enhancing the performance of the model. Over the years, researchers have applied different meta-heuristic optimization techniques for the purpose of FS as these overcome the limitations of traditional optimization approaches. Going by the trend, this paper introduces a novel FS technique based on the Enhanced LFM on BDA meta-heuristic algorithm, which was recently developed. As a wrapper-based technique for identifying the optimal subset of features for a certain classification assignment in this article. The Enhanced Levy Flight Mechanism on BDA is an upgraded version of the Binary Dragonfly Algorithm (BDA) in which the Levy Flight Mechanism (LFM) has been modified to improve the algorithm's randomisation, probabilistic behaviour, and artificial dragonfly. The suggested Enhanced LFM is pitted against nine other algorithms on BDA. The performance of eighteen datasets from the University of California, Irvine (UCI) collection is evaluated and compared using a variety of assessment measures. Using Enhanced LFM on BDA enhances prediction accuracy while lowering the number of selected criteria, according to the findings.

I. INTRODUCTION

Over the last decade, the number of dimensions in datasets has drastically expanded due to the fast growth of information science [1, 2]. Some dataset characteristics are recognised to be unnecessary and redundant information, according to [3], and these features might significantly influence the learning algorithm's effectiveness. To overcome this problem, possible dimension reduction methods are necessary.

In the data mining process, feature selection is often thought of as a pre-processing technique for finding an optimal subset of features from a given collection of all features. Its goal is to minimise the machine learning model's complexity, minify undesired information (irrelevant and redundant features), and improve prediction accuracy [4]. Pre-processing algorithms of this type include the search method and feature subset assessment are the two key processes. The former use certain strategies to pick a subset of characteristics, whilst the later uses a classifier to assess the quality of the subset [5]. There are two types of feature selection approaches in the literature: filter-based and wrapper-based. The data-reliant requirements are used in the filter-based procedure to evaluate the merits of the feature subset. The wrapper-based strategy, on the one hand, used a specialised classifier to estimate the relevant characteristics [6]. The filter-based technique takes less time than the wrapper-based one. This is because the feature selection procedure does not employ a machine learning algorithm. The wrapper-based strategy, on the other hand, may frequently locate the optimal feature subset that conveys the original features[6,7].

Metaheuristic algorithms have been recognised as the most efficient and reliable optimisation approaches in the last decade, particularly when dealing with difficult, high-dimensional issues. These methods have been frequently employed in real-world issues to increase performance [5]. Particle swarm optimisation [8], genetic algorithm [9], water cycle algorithm [10], animal migration optimisation [11], forest optimisation algorithm [12], human learning algorithm [13], heat transfer search [14], lightning search algorithm [15], monarch butterfly optimisation [16], and dragonfly algorithm [17] are some examples of metaheuristic algorithms. Metaheuristic algorithms have recently been effectively applied to a variety of technical and clinical challenges, including optimal neural network connection weights [18], numerical optimisation [19], cloud computing [20], stock market index prediction [21], and so on and estimate of solar photovoltaic (PV) parameters [22]

Researchers use metaheuristic algorithms (wrapper-based approaches) to handle feature selection issues because they are strong and perform well. To overcome feature selection challenges in classification tasks, Zhang et al. [23] suggested a Gaussian mutational chaotic fruit fly optimisation technique. A crow search algorithm was refined to pick prospective characteristics for enhancing Parkinson's disease diagnosis [24]. The paper introduced a new fitness function combined with a random forest classifier to improve the chaotic crow search algorithm's performance. Due to the high complexity of the random forest model, the suggested technique may take additional processing time when applied to high-dimensional datasets.

The Dragonfly Algorithm (DA) is a new algorithm inspired by the natural swarming patterns of dragonflies. It was created to tackle continuous optimisation problems. Mirjalili later devised a binary version of the dragonfly algorithm (BDA) for solving discrete issues (such as feature selection) [17]. Although BDA has demonstrated superior performance in various benchmark tests, its widespread use may result in the local optima issue. As a result, BDA might not be able to locate the worldwide best solution [25]. As a result, many performance-enhancing tactics may be introduced into BDA.

This study proposes a novel FS technique based on the Enhanced LFM on BDA meta-heuristic algorithm, which was recently developed. As a wrapper-based technique for identifying the optimal subset of features for a certain classification assignment in this article. The Enhanced Levy Flight Mechanism on BDA is an upgraded version of the Binary Dragonfly Algorithm (BDA) in which the Levy Flight Mechanism (LFM) has been modified to improve the algorithm's randomisation, probabilistic behaviour, and artificial dragonfly. The suggested Enhanced LFM is pitted against nine other algorithms on BDA. The performance of eighteen datasets from the University of California, Irvine (UCI) collection is evaluated and compared using a variety of assessment measures. Using Enhanced LFM on BDA enhances prediction accuracy while lowering the number of selected criteria, according to the findings.

II. RELATED WORK

As part of the popular technologies in information areas, data mining and data analysis have established the potential value of big data in the training process [26]. In recent years, many feature selection algorithms have been proposed and adopted for diagnosis, classification, categorization, pattern recognition, and detection [27–33]. For filter-based feature selection, Xue et al. [34] proposed a robust re-weighted multi-view feature selection algorithm. The proposed model allowed multiple relevant views as well as the specificity of each view in the process of selection. In [35], a feature selection model associated with a radial basis function network was designed to rank the importance of features. Kanimozhi and Manjula [36] proposed an incremental filtering feature selection approach for effective classification. They integrated a fuzzy rough set for assessing high-quality features. Another class discriminative degree-based feature selection approach was proposed in [37] for medical diagnosis.

Among the wrapper-based methods, particle swarm optimization (PSO) and its binary variant (BPSO) have been widely used in feature selection works. The self-adaptive parameter and strategy based PSO [38] and improved BPSO [39] were two recent approaches implemented in wrapper feature selection. Lately, authors in [40] introduced a multi-swarm mechanism to separate all the particles into several small size sub-swarms, and then a local version PSO was used to progress those sub-swarms. Besides, the proposed PSO approach utilized an elite learning strategy that allowed the exchange of information between sub-swarms, which can effectively improve the diversity of the population.

Another popular and well-known wrapper method was the genetic algorithm (GA). The GA algorithm solves optimization problems using principles of evolution in nature: selection, crossover, and mutation [41]. However, GA is a stochastic algorithm and requires tuning to ensure its maximum performance [42]. Therefore, Kromer et al. [43] designed a new crossover operator according to the Mergesort algorithm, which

enabled the recombination of valid solutions and avoided invalid resolutions. Later on, Too and Abdullah [44] proposed a rival genetic algorithm and its fast version for feature selection problems. The rival genetic algorithm first divided the chromosomes into winner and loser groups, and then the parents were selected based on the tournament selection for crossover operation. Afterward, the mutation was performed using a dynamic mutation operator. Their study reported the proposed approaches can perform faster and overtake other conventional methods. However, the algorithm was complicated due to its complex structure.

In [4], Hancer et al. proposed a binary artificial bee colony based on the Jaccard coefficient dissimilarity. The authors borrowed a neighborhood mechanism from the differential evolution algorithm and integrated it into the similarity-based search to expand the global search capability. They reported that the proposed approach can usually overtake other traditional methods in feature subset selection. Nevertheless, the proposed approach was getting complicated because of its complex structure. Moreover, Wan et al. [45] modified the binary coded ant colony optimization with pheromone and visibility density models for feature selection problems. They performed the hybridization of binary ant colony optimization and genetic algorithm within two models to facilitate the searching behavior and improve the robustness of the algorithm. However, the proposed algorithm only validated on low dimensional datasets with the number of features less than 100.

An improved harris hawk optimizer (IHHO) based feature selection has been developed by [46]. The study performed a hybridization of harris hawk optimizer and salp swarm algorithm to enhance the searching behavior during optimization. In [47], the chaotic atom search optimization that used twelve different chaotic maps were proposed. The authors employed the chaotic map to alter the multiplier weight in order to boost the convergence behavior. Moreover, two variants of atom search optimization based feature selection approaches were proposed in [48, 49]. Another feature selection study that employed the ant colony optimization, artificial bee colony and GA algorithm for keystroke dynamic was presented in [50].

Grey wolf optimizer (GWO) is a recent metaheuristic algorithm that mimics the leadership hierarchy and hunting behavior of grey wolves in nature [51]. Lately, the GWO and its binary variants (BGWO) have been used as the wrapper method for feature selection in [52, 53]. Their studies showed that the proposed GWO variants outperformed other optimizers. Moreover, Chantar et al. [54] developed an elite crossover mechanism into BGWO for Arabic text classification. In addition, Too and Abdullah [55] implemented the opposition based learning into competitive grey wolf optimizer to improve the prediction accuracy of hand motions classification. Furthermore, authors in [56] proposed a fusion of grey wolf optimizer and biphasic mutation operator to evolve the character of the leader during the search process

Another inspiration based on the whale optimization algorithm (WOA) was developed for feature selection. As an example, Mafarja and Mirjalili [57] proposed WOA based feature selection approach in which the tournament selection was employed to replace the random mechanism in the selection process. In [58], Mafarja and Mirjalili hybridized the WOA with a simulated annealing algorithm, which strengthened the capability of the algorithm in exploring the feature space. Moreover, Tubishat et al. [59] proposed an improved WOA to solve the feature selection issue in Arabic sentiment analysis. They first applied the information gain to filter some unwanted features, and then a novel approach was introduced based on oppositionbased learning principles to generate the new opposite solutions.

In COVID-19 patient detection, Shaban et al. [60] introduced a hybrid feature selection methodology to select the optimal features extracted from the chest computed tomography (CT) images. They first filtered some unwanted features and then accurately selected the informative feature subset using GA. Jain et al. [61] proposed a deep network model to identify the COVID19 disease based on X-ray images. The proposed model was reliable and accurate, however, it heavy relies on the stage1 model [61]. To effectively remove the negative features from COVID-19 prediction, Tuncer et al. [62] designed an iterative ReliefF based feature selection algorithm. Moreover, the authors in [63] applied six popular machine learning algorithms for forecasting one, three, and six days-ahead the COVID-19 cumulative confirmed cases in ten Brazilian states. Another recent COVID-19 study was presented in [64].

The dragonfly algorithm (DA) was recently developed by Mirjalili for global optimization purpose. He then proposed a V-shaped transfer function and integrated it into DA to permit the dragonflies for searching on the binary search space [17]. In [65], the authors employed the binary dragonfly algorithm (BDA) for feature selection. However, like other metaheuristic algorithms, the BDA also has the limitation of restricting the local optima. Therefore, Sayed et al. [25] integrated the chaos theorem into BDA to boost the convergence rate. Among ten different chaotic maps, they reported that the Gauss map was more capable of enhancing the performance of learning model. Moreover, an improved BDA approach was designed for infant cry classification. The proposed model applied the elitism mechanism that stored the potential dragonfly and then inserted it to replace the worst dragonfly in the population, which enhanced the convergence behavior of the algorithm in finding the promising solutions.

III. DRAGONFLY ALGORITHM

III.I OPERATORS FOR EXPLORATION AND EXPLOITATION

According to Reynolds, the behaviour of swarms follows three primitive principles [66]

- Separation, which refers to the static collision avoidance of the individuals from other individuals in the neighbourhood.
- Alignment, which indicates velocity matching of individuals to that of other individuals in neighbourhood.
- Cohesion, which refers to the tendency of individuals towards the centre of the mass of the neighbourhood.

The main objective of any swarm is survival, so all of the individuals should be attracted towards food sources and distracted outward enemies. Considering these two behaviours, there are five main factors in position updating of individuals in swarms as shown in Fig. 1.

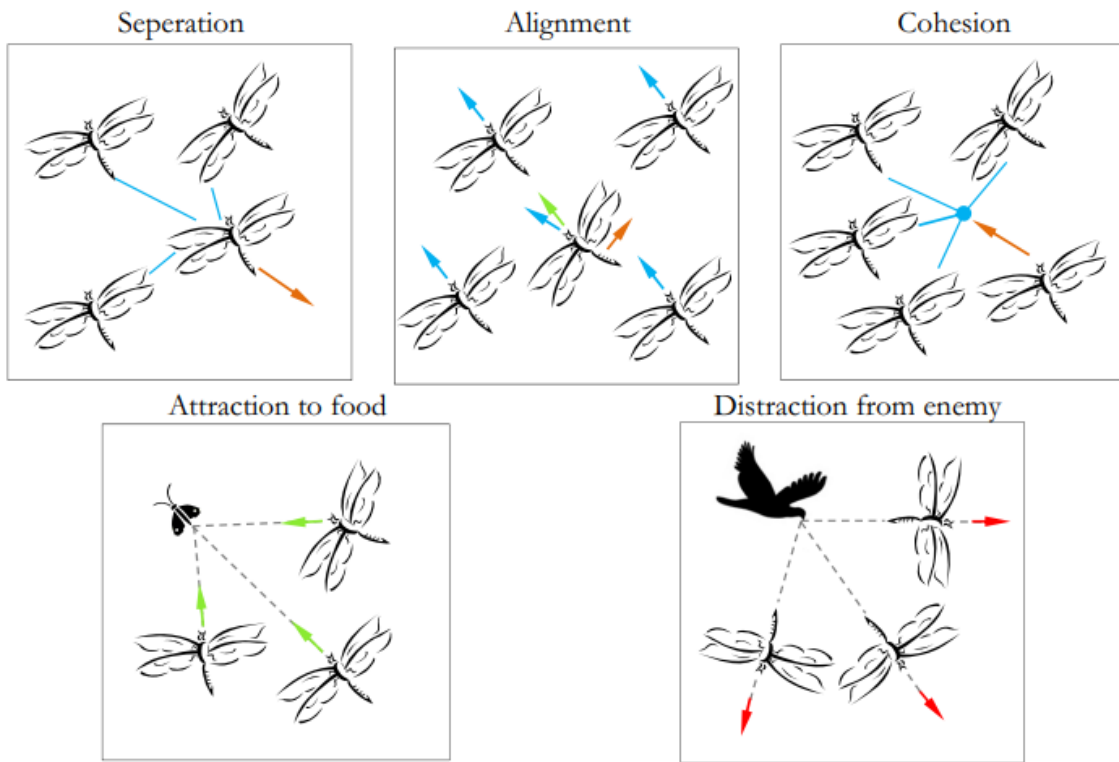


Fig 1. Primitive corrective patterns between individuals in a swarm

Each of these behaviours is mathematically modelled as follows:

The separation is calculated as follows [66]:

$$S_i = - \sum_{j=1}^N X - X_j \quad (1)$$

where X is the position of the current individual, X_j shows the position j -th neighbouring individual, and N is the number of neighbouring individuals.

Alignment is calculated as follows:

$$A_i = \frac{\sum_{j=1}^N V_j}{N} \quad (2)$$

where X_j shows the velocity of j -th neighbouring individual.

The cohesion is calculated as follows:

$$C_j = \frac{\sum_{j=1}^N X_j}{N} - X \quad (3)$$

where X is the position of the current individual, N is the number of neighbourhoods, and X_j shows the position j -th neighbouring individual.

Attraction towards a food source is calculated as follows:

$$F_i = X^+ - X \quad (4)$$

where X is the position of the current individual, and X^+ shows the position of the food source.

Distraction outwards an enemy is calculated as follows:

$$E_i = X^- + X \quad (5)$$

where X is the position of the current individual, and X^- shows the position of the enemy.

The behaviour of dragonflies is assumed to be the combination of these five corrective patterns in this paper. To update the position of artificial dragonflies in a search space and simulate their movements, two vectors are considered: step (ΔX) and position (X). The step vector is analogous to the velocity vector in PSO, and the DA algorithm is developed based on the framework of the PSO algorithm. The step vector shows the direction of the movement of the dragonflies and defined as follows (note that the position updating model of artificial dragonflies is defined in one dimension, but the introduced method can be extended to higher dimensions):

$$\Delta X_{t+1} = (sS_i + aA_i + cC_i + fF_i + eE_i) + w\Delta X_t \quad (6)$$

where s shows the separation weight, S_i indicates the separation of the i -th individual, a is the alignment weight, A is the alignment of i -th individual, c indicates the cohesion weight, C_i is the cohesion of the i -th individual, f is the food factor, F_i is the food source of the i -th individual, e is the enemy factor, E_i is the position of enemy of the i -th individual, w is the inertia weight, and t is the iteration counter.

After calculating the step vector, the position vectors are calculated as follows:

$$X_{t+1} = X_t + \Delta X_{t+1} \quad (7)$$

where t is the current iteration. With separation, alignment, cohesion, food, and enemy factors (s , a , c , f , and e), different explorative and exploitative behaviours can be achieved during optimization. Neighbours of dragonflies are very important, so a neighbourhood (circle in a 2D, sphere in a 3D space, or hypersphere in an n D space) with a certain radius is assumed around each artificial dragonfly. An example of swarming behaviour of dragonflies with increasing neighbourhood radius using the proposed mathematical model is illustrated in Fig. 2.

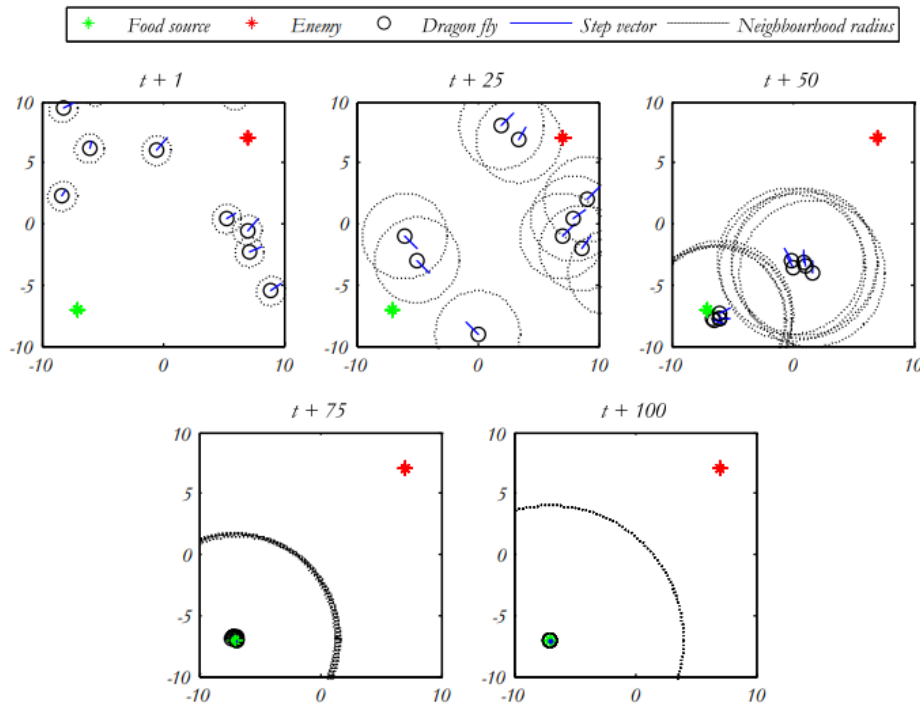


Fig 2:Swarming behaviour of artificial dragon flies

As discussed in the previous subsection, dragonflies only show two types of swarms: static and dynamic as shown in Fig. 3. As may be seen in this figure, dragonflies tend to align their flying while maintaining proper separation and cohesion in a dynamic swarm. In a static swarm, however, alignments are very low while cohesion is high to attack preys. Therefore,

we assign dragonflies with high alignment and low cohesion weights when exploring the search space and low alignment and high cohesion when exploiting the search space. For transition between exploration and exploitation, the radii of neighbourhoods are increased proportional to the number of iterations. Another way to balance exploration and exploitation is to adaptively tune the swarming factors (s, a, c, f, e, and w) during optimization.

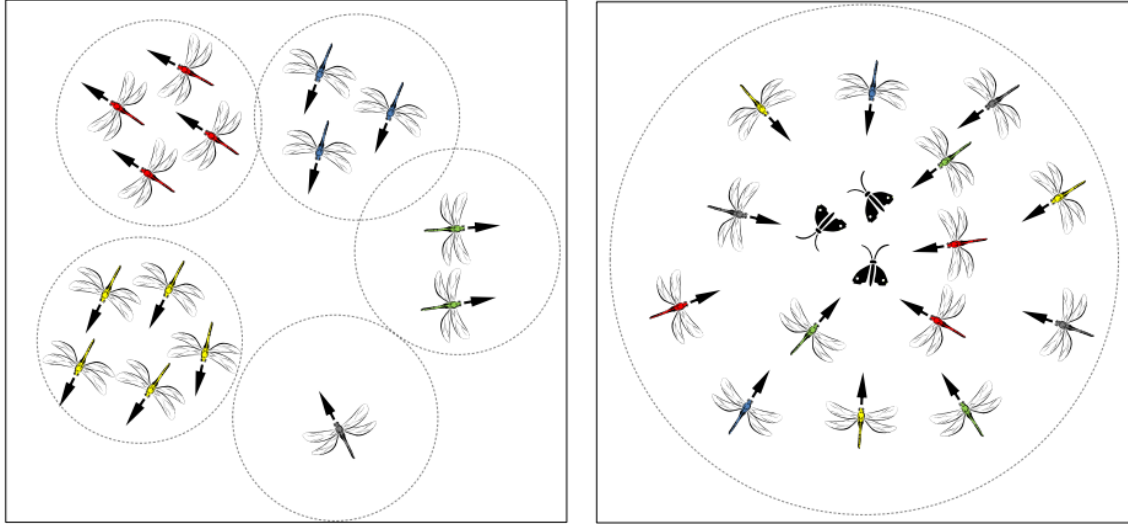


Fig 3: Dynamic versus static dragonfly swarms

To improve the randomness, stochastic behaviour, and exploration of the artificial dragonflies, they are required to fly around the search space using a random walk (Le'vy flight) when there is no neighbouring solutions. In this case, the position of dragonflies is updated using the following equation:

$$X_{t+1} = X_t + Levy(d) \times X_t \quad (8)$$

where t is the current iteration, and d is the dimension of the position vectors.

The Levy flight is calculated as follows [67]:

$$Levy(x) = 0.01 \times \frac{r1 \times \sigma}{|r2|^{\frac{1}{b}}} \quad (9)$$

where r1, r2 are two random numbers in [0,1], b is a constant (equal to 1.5 in this work), and r is calculated as follows:

$$\sigma = \left(\frac{\Gamma(1+\beta) \times \sin(\frac{\pi\beta}{2})}{\Gamma(\frac{1+\beta}{2}) \times \beta \times 2^{\frac{\beta-1}{2}}} \right)^{\frac{1}{\beta}} \quad (10)$$

Where $\Gamma(x) = (x - 1)!$

III.II THE DA ALGORITHM FOR SINGLE-OBJECTIVE PROBLEMS

The DA algorithm starts optimization process by creating a set of random solutions for a given optimization problems. In fact, the position and step vectors of dragonflies are initialized by random values defined within the lower and upper bounds of the variables. In each iteration, the position and step of each dragonfly are updated using Eqs. (7)/(8) and (6). For updating X and DX vectors, neighbourhood of each dragonfly is chosen by calculating the Euclidean distance between all the dragonflies and selecting N of them. The position updating process is continued iteratively until the end criterion is satisfied. The pseudo-codes of the DA algorithm are provided in Fig. 4.

It is worth discussing here that the main differences between the DA and PSO algorithm are the consideration of separation, alignment, cohesion, attraction, distraction, and random walk in this work. Although there are some works in the literature that attempted to integrate separation, alignment, and cohesion to PSO [68–70], this paper models the swarming behaviour of dragonflies by considering all the possible factors applied to individuals in a swarm. The concepts of static and dynamic swarms are quite novel as well. The proposed model of this work is also completely different from the current improved PSO in the literature cited above.

Algorithm 1. Pseudo-code of the DA algorithm
<i>Initialize the dragonflies population X_i ($i = 1, 2, \dots, n$)</i> <i>Initialize step vectors ΔX_i ($i = 1, 2, \dots, n$)</i> while the end condition is not satisfied <i>Calculate the objective values of all dragonflies</i> <i>Update the food source and enemy</i> <i>Update w, s, a, c, f, and e</i> <i>Calculate S, A, C, F, and E using Eqs. (1) to (5)</i> <i>Update neighbouring radius</i> if a dragonfly has at least one neighbouring dragonfly <i>Update velocity vector using Eq. (6)</i> <i>Update position vector using Eq. (7)</i> else <i>Update position vector using Eq. (8)</i> end if <i>Check and correct the new positions based on the boundaries of variables end while</i> end while

Fig. 4 Pseudo-codes of the DA algorithm

III.III THE DA ALGORITHM FOR SINGLE-OBJECTIVE PROBLEMS

Optimization in a binary search space is very different than a continuous space. In continuous search spaces, the search agents of DA are able to update their positions by adding the step vectors to the position vectors. In a binary search space, however, the position of search agents cannot be updated by adding step vectors to X since the position vectors of search agents can only be assigned by 0 or 1. Due to the similarity of DA and other SI techniques, the current methods for solving binary problems in the literature are readily applicable to this algorithm.

According to Mirjalili and Lewis [71], the easiest and most effective method to convert a continuous SI technique to a binary algorithm without modifying the structure is to employ a transfer function. Transfer functions receive velocity (step) values as inputs and return a number in $[0,1]$, which defines the probability of changing positions. The output of such functions is directly proportional to the value of the velocity vector. Therefore, a large value for the velocity of a search agent makes it very likely to update its position. This method simulates abrupt changes in particles with large velocity values similarly to continuous optimization (Fig. 6). Two examples of transfer functions in the literature are illustrated in Fig. 6 [71–73].

As may be seen in this figure, there are two types of transfer functions: s-shaped versus v-shaped. According to Saremi et al. [72], the v-shaped transfer functions are better than the s-shaped transfer functions because they do not force particles to take values of 0 or 1. In order to solve binary problems with the BDA algorithm, the following transfer function is utilized [73]:

$$T(\Delta x) = \left| \frac{\Delta x}{\sqrt{\Delta x^2 + 1}} \right| \quad (11)$$

This transfer function is first utilized to calculate the probability of changing position for all artificial dragonflies. The following new updating position formula is then employed to update the position of search agents in binary search spaces:

$$X_{t+1} = \begin{cases} \neg X_t & r < T(\Delta x_{t+1}) \\ X_t & r \geq T(\Delta x_{t+1}) \end{cases} \quad (12)$$

where r is a number in the interval of $[0,1]$.

With the transfer function and new position updating equations, the BDA algorithm will be able to solve binary problems easily subject to proper formulation of the problem. It should be noted here that since the distance of dragonflies cannot be determined in a binary space as

clearly as a continuous space, the BDA algorithm considers all of the dragonflies as one swarm and simulate exploration/exploitation by adaptively tuning the swarming factors (s , a , c , f , and e) as well as the inertia weigh (w). The pseudo-codes of the BDA algorithm are presented in Fig. 6

Algorithm 2. Pseudo-code of the BDA algorithm
<i>Initialize the dragonflies population X_i ($i = 1, 2, \dots, n$)</i> <i>Initialize step vectors ΔX_i ($i = 1, 2, \dots, n$)</i> while Maximum number of iterations is not met <i>Evaluate the fitness values of dragonflies</i> <i>Update the food source, X_f and enemy, X_e</i> <i>Update s, a, c, f, e, and w</i> for $i = 1$ to number of dragonflies, N <i>Calculate S, A, and C using (1), (2), and (3)</i> <i>Compute F and E using (4) and (5)</i> <i>Update step vector using (6)</i> <i>Update the position of dragonfly (i-th) using (11)</i> end for end while

Fig. 5 Pseudo-codes of the BDA algorithm

IV. THE PROPOSED ALGORITHM

FS is a binary optimisation issue with only binary 0 and 1 values for solutions. As a consequence, the binary variant of the DA approach is suitable for dealing with this problem. In this work, the answer to the problem is represented by a vector of zeros and ones, with the zero element indicating that the relevant characteristic is not picked and the one element indicating that it is. The length of the solution vector is defined by the number of features in the original dataset.

A wrapper feature selection technique built on the BDA algorithm is suggested in this study. The voting classifier was used to enhance the Binary Dragonfy Optimizer's best solution (subset of features) thus far, with the estimators being logistic regression, decision tree, and SVM, as well as a change to the Levy Flight Mechanism (LFM) i.e. equation 13.

This section introduces and applies a novel binary dragonfly technique to feature selection.

Paul Levy, a French mathematician, was the inspiration for the LFM. This system has an unlimited number of variables (possible length).

When a neighbourhood solution is not available, a random walk (LFM) solution is utilised to promote randomisation, probabilistic behaviour, and artificial route discovery. As a consequence, as shown in equations (8), (9) and (10) accordingly, the locations of fake dragonflies are updated.

In the LFM utilised in DA, a multiplication was performed that was not contained in the flying technique's original mathematical calculation. This multiplication was created by taking 1% of the LFM size, as shown in equation (9).

The objective was to keep the step size in check. Following LFM, this multiplication yields the value of a solution, which is the optimum individual deviation amount (position of the best individual).

Although LFM enhances DA performance to some extent, it has the disadvantage of creating very long steps depending on the parameters of the mechanism. The step size, as illustrated in equation, is used to try to regulate these crucial aspects of the algorithm (13). This approach is preferable than the prior one.

$$Levy(x) = \left(i/N^2\right)^{(i+N)^{1/2}} \times \frac{r1 \times \sigma}{|r2|^{\frac{1}{\beta}}} \quad (13)$$

Where I is the current iteration, N is the total number of iterations, r1, r2 are two random values in the range [0,1], b is a constant (in this case, 1.5), and r is computed as follows:

$$\sigma = \left(\frac{\Gamma(1+\beta) \times \sin(\frac{\pi\beta}{2})}{\Gamma(\frac{1+\beta}{2}) \times \beta \times 2^{(\frac{\beta-1}{2})}} \right)^{\frac{1}{\beta}}$$

Where $\Gamma(x) = (x - 1)!$

Algorithm 3. Pseudo-code of the Enhanced LFM on BDA
<i>Initialize the dragonflies population X_i ($i = 1, 2, \dots, n$)</i> <i>Initialize step vectors ΔX_i ($i = 1, 2, \dots, n$)</i> while Maximum number of iterations is not met <i>Evaluate the fitness values of dragonflies</i> <i>Update the food source, X_f and enemy, X_e</i> <i>Update s, α, c, f, e, and w</i> for $i = 1$ to number of dragonflies, N <i>Calculate S, A, and C using (1), (2), and (3)</i> <i>Compute F and E using (4) and (5)</i> <i>Update neighbouring radius</i> if a dragonfly has at least one neighbouring dragonfly <i>Update velocity vector using Eq. (6)</i> <i>Update position vector using Eq. (7)</i> else <i>Update position vector using Eq.(13)</i> end if end for end while

Fig. 6 Pseudo-codes of the enhanced LFM on BDA algorithm

V. EXPERIMENTAL RESULTS AND DISCUSSIONS

A. DATA SETS

Table I lists the 18 datasets that we used to assess the effectiveness of the suggested approach. They are well-known benchmark datasets from the data repository at the University of California, Irvine [52]. Using the same data sets, several scholars investigate alternative feature selection methodologies. The data sets utilised to assess the proposed Enhanced LFM on Binary Dragon algorithm-based feature selection technique are listed in Table 1.

Data set	Features	Instances
Breastcancer	9	699
BreastEW	30	569
CongressEW	16	435
Exactly	13	1000
Exactly2	13	1000
HeartEW	13	270
IonosphereEW	34	351
KrvskpEW	36	3196
Lymphography	18	148
M-of-n	13	1000
PenglungEW	325	73
SonarEW	60	208
SpectEW	22	267
Tic-tac-toe	9	958
Vote	16	300
WaveformEW	40	5000
WineEW	13	178
Zoo	16	101

Table 1 Datasets used in this experiment

B. PARAMETER SETTING

Parameter	Value
Number of dragonflies, N	10
Maximum iterations, T	100
Number of dimensions, D	Number of features in each dataset
Personal learning rate, pl	0.5
Global learning rate, gl	0.8
Search dimension	$[-10,10] \times \text{Number of dimensions}$

Table 2 Parameter setting

C. FITNESS EVALUATION

Using a pre-defined goal function, each dragonfly is analysed iteratively. The objective function is used to assess the quality of the solutions [74]. In this article, the goal function is established to maximise classification accuracy while reducing the number of given criteria. [75, 76] The following is the objective function:

$$Fitness = \alpha \gamma_R(D) + \beta \frac{|R|}{|C|} \quad (14)$$

where $\alpha \gamma_R(D)$ is the classification error rate of a specific classifier (in this case, the voting classifier). In addition, $|R|$ is the cardinality of the selected subset, $|C|$ is the total number of features in the dataset, α and β are two parameters related to the significance of classification quality and subset length, and $\alpha \in [0, 1]$ and $\beta = (1 - \alpha)$ are borrowed from Eq. 6.

D. EVALUTION METRICS

The datasets are partitioned into three parts at random (validation, training, and testing datasets). The data separation method is done several times to ensure that the results are strong and quantifiable. There are a total of 20 runs, each of which uses the validation data to look at the following statistical metrics:

1. THE AVERAGE OF CLASSIFICATION ACCURACY:

It's a statistic that shows how accurate the classifier is given a set of attributes after N algorithm runs, and it's calculated as follows:

$$AvgAcc = \frac{1}{N} \sum_{i=1}^N AvgAcc^i$$

Where $AvgAcc^i$ is the value of accuracy gained at run i.

2. THE AVERAGE OF SELECTED FEATURE :

When the algorithm is run N times, it produces a statistic that compares the average of selected attributes to the entire number of features.

$$AvgSelection = \frac{1}{N} \sum_{i=1}^N \frac{AvgSelection^i}{M}$$

Where M is the total no.of features in data set and $AvgSelection^i$ is the no.of features selected at run i.

3. THE MEAN FITNESS FUNCTION:

It's a figure that shows the average fitness value achieved at run I after the algorithm has been run N times.

$$Mean = \frac{1}{N} \sum_{i=1}^N m^i$$

Where m^i is the mean fitness gained at run i.

4. THE BEST FITNESS FUNCTION:

It's a metric that shows the lowest fitness value achieved at run I after the algorithm has been run N times.

$$Mean = \min(m)$$

Where m is the collection of best fitness over N runs.

5. THE WORST FITNESS FUNCTION:

It's a statistic that shows the highest level of fitness at run I after running the algorithm N times.

$$Mean = \max(m)$$

Where m is the collection of best fitness over N runs.

6. AVERAGE COMPUTATIONAL TIME:

It's a number that reflects the average computation time in seconds obtained after running an algorithm N times.

$$AvgCT = \frac{1}{N} \sum_{i=1}^N AcgCT^i$$

Where $AvgCT^i$ is the time obtained at run i.

E. RESULT AND DISCUSSION

This section contains all of the documented outcomes from the proposed FS method. When the average of selected features by enhanced LFM on BDA is smaller than BDA, it signifies that improved LFM on BDA deleted some noisy or unnecessary characteristics picked by

BDA. Since the main goal of this work is to improve the performance of the Binary Dragonfy Optimizer by using a voting classifier wherein the estimators are logistic regression, decision tree, and SVM, as well as changes to the Levy Flight Mechanism (equation 13) improve the best solution (subset of features) discovered thus far., we performed further statistical research to show that the enhanced LFM on BDA strategy is superior to utilising the basic BDA alone to solve the feature selection problem.

Average Accuracy								
Dataset	GA	PSO	DF	BPSO	BALO	BGWO	BDA_SA	Enhanced LFM on BDA
BreastCancer	0.974	0.97	0.96	0.949	0.931	0.96	0.988	0.98
BreastCancerEW	0.967	0.964	0.961	0.924	0.93	0.94	0.974	0.98
CongressEW	0.967	0.964	0.967	0.912	0.915	0.92	0.975	0.97
Exactly	0.717	0.714	0.98	0.672	0.626	0.67	1	0.74
Exactly2	0.742	0.749	0.745	0.725	0.702	0.72	0.759	0.76
HeartEW	0.853	0.829	0.83	0.789	0.751	0.75	0.895	0.86
IonosphereEW	0.853	0.862	0.93	0.845	0.813	0.79	0.929	0.92
KrVsKpEW	0.953	0.953	0.953	0.85	0.761	0.86	0.976	0.98
Lymphography	0.848	0.82	0.877	0.73	0.684	0.79	0.911	0.92
M-of-n	0.892	0.927	0.992	0.814	0.733	0.8	0.997	1
PengLung	0.794	0.756	0.895	0.728	0.789	0.72	0.93	0.86
Sonar	0.843	0.831	0.915	0.799	0.778	0.8	0.917	0.92
SpectEW	0.865	0.856	0.853	0.808	0.831	0.79	0.866	0.875
Tic-Tac-Toe	0.765	0.769	0.788	0.712	0.694	0.71	0.818	0.77
Vote	0.96	0.96	0.958	0.905	0.882	0.92	0.964	0.98
WaveForm	0.782	0.784	0.75	0.743	0.701	0.74	0.805	0.87

Table 3 Average classification accuracy

Table 3 displays average classification accuracy, which is generally high for almost half of collected data sets except BreastCancer, CongressEW, Exactly, HeartEW, IonosphereEW, PengLung, Tic-Tac-Toe which means the proposed work has best performance than other compared works.

Mean Features								
Dataset	GA	PSO	DF	BPSO	BALO	BGWO	BDA_SA	Enhanced LFM on BDA
BreastCancer	0.511	0.511	0.55	6	6	4	6	0.5
BreastCancerEW	0.46	0.44	0.395	12.8	22.6	20.6	12.45	0.5
CongressEW	0.362	0.4	0.288	3.2	9.3	9.5	4.15	0.5
Exactly	0.523	0.492	0.469	6.2	7.7	7.75	6	0.4
Exactly2	0.215	0.462	0.208	2.05	8.45	8.2	1.45	0.4
HeartEW	0.508	0.538	0.527	3.55	8.9	8.5	6.9	0.38
Ionosphere	0.418	0.424	0.338	12.3	25.5	20.6	10.4	0.44
KrVsKpEW	0.539	0.533	0.493	16.85	26.9	24.3	14.4	0.3
Lymphography	0.4	0.4	0.453	6.05	11.05	11.8	8.5	0.46
M-of-n	0.677	0.6	0.465	6	7.5	7.65	6.15	0.4
PengLung	0.434	0.387	0.38	146.25	253.65	205	141.3	0.5
Sonar	0.42	0.45	0.458	27	47.45	40.1	23.9	0.4
SpectEW	0.373	0.464	0.361	8	15.35	13.8	8.85	0.5
Tic-Tac-Toe	0.556	0.556	0.661	8.9	9	6	7.8	0.44
Vote	0.362	0.512	0.259	5	8.5	8.1	3.05	0.5
WaveForm	0.58	0.52	0.524	21.15	32.4	30.6	21.1	0.5
Wine	0.477	0.523	0.485	6.45	10.25	8.4	8.85	0.46

Table 4 Average Feature Selection

Table 4 displays the feature selection averages for the various FS approaches on each more robust data set. When compared to numerous well-known methods in [77-79], the number of characteristics picked using this technique is fairly minimal.

Mean Fitness				
Dataset	GA	PSO	DF	Enhanced LFM on BDA
BreastCancer	0.026	0.03	0.038	0.02
BreastCancerEW	0.033	0.036	0.04	0.03
CongressEW	0.033	0.036	0.035	0.04
Exactly	0.283	0.286	0.006	0.3
Exactly2	0.258	0.251	0.249	0.24
HeartEW	0.147	0.171	0.167	0.17
Ionosphere	0.147	0.138	0.071	0.09
KrVsKpEW	0.047	0.047	0.031	0.02
Lymphography	0.152	0.18	0.125	0.15
M-of-n	0.108	0.073	0.005	0.12
PengLung	0.206	0.244	0.089	0.08
Sonar	0.157	0.169	0.066	0.15
SpectEW	0.135	0.144	0.144	0.13
Tic-Tac-Toe	0.235	0.231	0.209	0.2
Vote	0.04	0.04	0.041	0.04
WaveForm	0.218	0.216	0.244	0.15
Wine	0.02	0.023	0.02	0.015
Zoo	0.103	0.099	0.039	0.01

Table 5 Mean Fitness

On each dataset, Table 5 displays the mean fitness for the used FS approach. In comparison to other techniques, the mean fitness is relatively low. The proposed variant also had the best mean fitness results

Best Fitness								
Dataset	GA	PSO	DF	BPSO	BALO	BGWO	BDA_SA	Enhanced LFM on BDA
BreastCancer	0.01	0.02	0.03	0.038	0.038	0.03	0.018	0.01
BreastCancerEW	0.021	0.026	0.025	0.044	0.036	0.05	0.03	0.01
CongressEW	0.021	0.014	0.021	0.034	0.045	0.03	0.028	0.03
Exactly	0.258	0.189	0.005	0.037	0.164	0.16	0.005	0.2
Exactly2	0.234	0.231	0.226	0.243	0.257	0.25	0.24	0.24
HeartEW	0.089	0.133	0.145	0.135	0.165	0.17	0.109	0.13
Ionosphere	0.12	0.111	0.042	0.113	0.141	0.18	0.074	0.07
KrVsKpEW	0.032	0.036	0.022	0.03	0.045	0.05	0.028	0.02
Lymphography	0.14	0.14	0.072	0.145	0.156	0.11	0.092	0.07
M-of-n	0.084	0.018	0.005	0.005	0.046	0.04	0.008	0
PengLung	0.12	0.16	0.031	0.165	0.122	0.22	0.049	0.02
Sonar	0.1	0.086	0.034	0.093	0.123	0.13	0.086	0.02
SpectEW	0.079	0.101	0.107	0.134	0.096	0.13	0.137	0.12
Tic-Tac-Toe	0.204	0.204	0.182	0.201	0.217	0.19	0.189	0.22
Vote	0.03	0.01	0.018	0.032	0.062	0.05	0.037	0.02
WaveForm	0.202	0.202	0.228	0.2	0.219	0.22	0.198	0.13
Wine	0.017	0.017	0.005	0.018	0.008	0.04	0.008	0
Zoo	0.063	0.069	0.001	0.045	0.033	0.08	0.024	0

Table 6 Best Fitness

The best fitness for each data set is shown in Table 6. In terms of fitness, the recommended technique delivered the best results. The greatest fitness is very modest in compared to other strategies, and it produced the best outcomes.

Worst Fitness				
Dataset	GA	PSO	DF	Enhanced LFM on BDA
BreastCancer	0.03	0.039	0.04	0.03
BreastCancerEW	0.047	0.05	0.049	0.04
CongressEW	0.048	0.062	0.048	0.01
Exactly	0.312	0.342	0.023	0.3
Exactly2	0.27	0.27	0.286	0.24
HeartEW	0.178	0.189	0.195	0.22
Ionosphere	0.171	0.162	0.094	0.12
KrVsKpEW	0.061	0.065	0.038	0.16
Lymphography	0.18	0.22	0.167	0.24
M-of-n	0.156	0.117	0.005	0.25
PengLung	0.32	0.32	0.165	0.22
Sonar	0.214	0.229	0.109	0.22
SpectEW	0.18	0.191	0.197	0.21
Tic-Tac-Toe	0.26	0.26	0.238	0.34
Vote	0.05	0.08	0.069	0.13
WaveForm	0.244	0.224	0.262	0.21
Wine	0.033	0.033	0.038	0.05
Zoo	0.152	0.152	0.139	0.09

Table 7 Worst Fitness

Table 7 displays the data sets with the worst fitness. When compared to other methodologies, the worst fitness for the vast majority of data sets is fairly low.

Dataset	Enhanced LFM on BDA
BreastCancer	0.01
BreastCancerEW	0.02
CongressEW	0.01
Exactly	0.03
Exactly2	0.04
HeartEW	0.012
Ionosphere	0.026
KrVsKpEW	0.22
Lymphography	0.19
M-of-n	0.03
PengLung	0.12
Sonar	0.03
SpectEW	0.19
Tic-Tac-Toe	0.02
Vote	0.02
WaveForm	0.5
Wine	0.189
Zoo	0.02

Table 8 Computation Time

The computation time for each data set is shown in Table 8. The time for all data sets, i.e. 18, is optimised. With the suggested update, the time metric is significantly lowered.

VI. CONCLUSION

An enhanced LFM on BDA was proposed and used to solve the problem of feature selection in this work. To confirm the effectiveness and the efficiency of the proposed method, 18 standard UCI benchmark datasets were used. A set of evaluation metrics were used to assess the proposed method. The proposed enhanced LFM on BDA was compared with a number of feature selection algorithms called GA [77], PSO [77], DF [77], BGWOPSO [78], WOSAT-2 [78], BPSO [79], BALO [79], BGWO [79], and BDA SA [79].

On the majority of datasets, the results showed that the suggested technique outperformed a wide variety of methods in terms of accuracy and number of features selected. Furthermore, a computational time was suggested that the enhanced LFM on BDA technique has revealed that the proposed method has a faster execution time. Furthermore, a comparison of statistical measures Mean, Best, Worst Fitness was undertaken, with the findings demonstrating that the suggested methodology outperformed existing state-of-the-art methods greatly. The suggested enhanced LFM on BDA method's better results show that it can regulate the trade-off between exploratory and exploitative tendencies throughout optimization rounds.

In future work, we recommend using the suggested upgraded LFM on BDA approach to address a variety of real-world issues, including engineering optimisation hurdles, scheduling

limitations, and/or molecular potential energy function. In addition, the suggested Feature Selection approach may be compared to other well-known classifiers, such as the Artificial Neural Network (ANN), to check if performance is constant or changes.

VII. REFERENCES

1. Han C, Zhou G, Zhou Y (2019) Binary Symbiotic Organism Search Algorithm for Feature Selection and Analysis. *IEEE Access* 7:166833–166859. <https://doi.org/10.1109/ACCESS.2019.2953800>
2. Quiroz JC, Banerjee A, Dascalu SM, Lau SL (2017) Feature Selection for Activity Recognition from Smartphone Accelerometer Data. *Intell Autom Soft Comput* 1–9. <https://doi.org/10.1080/10798587.2017.1342400>
3. Aljarah I, Habib M, Faris H, et al (2020) A dynamic locality multi-objective salp swarm algorithm for feature selection. *Comput Ind Eng* 147:106628. <https://doi.org/10.1016/j.cie.2020.106628>
4. Hancer E, Xue B, Karaboga D, Zhang M (2015) A binary ABC algorithm based on advanced similarity scheme for feature selection. *Appl Soft Comput* 36:334–348. <https://doi.org/10.1016/j.asoc.2015.07.023>
5. Arora S, Anand P (2019) Binary butterfly optimization approaches for feature selection. *Expert Syst Appl* 116:147–160. <https://doi.org/10.1016/j.eswa.2018.08.051>
6. Zhong C, Chen Y, Peng J (2020) Feature Selection Based on a Novel Improved Tree Growth Algorithm. *Int J Comput Intell Syst* 13:247–258. <https://doi.org/10.2991/ijcis.d.200219.001>
7. Al-Tashi Q, Kadir SJA, Rais HM, et al (2019) Binary Optimization Using Hybrid Grey Wolf Optimization for Feature Selection. *IEEE Access* 7:39496–39508. <https://doi.org/10.1109/ACCESS.2019.2906757>
8. Kennedy J (2011) Particle Swarm Optimization. In: *Encyclopedia of Machine Learning*. Springer, Boston, MA, pp 760–766
9. Holland JH (1992) Genetic Algorithms. *Sci Am* 267:66–73
10. Eskandar H, Sadollah A, Bahreininejad A, Hamdi M (2012) Water cycle algorithm – A novel metaheuristic optimization method for solving constrained engineering optimization problems. *Comput Struct* 110–111:151–166. <https://doi.org/10.1016/j.compstruc.2012.07.010>

11. Li X, Zhang J, Yin M (2014) Animal migration optimization: an optimization algorithm inspired by animal migration behavior. *Neural Comput Appl* 24:1867–1877. <https://doi.org/10.1007/s00521-013-1433-8>
12. Ghaemi M, Feizi-Derakhshi M-R (2014) Forest Optimization Algorithm. *Expert Syst Appl* 41:6676–6687. <https://doi.org/10.1016/j.eswa.2014.05.009>
13. Wang L, Yang R, Ni H, et al (2015) A human learning optimization algorithm and its application to multi-dimensional knapsack problems. *Appl Soft Comput* 34:736–743. <https://doi.org/10.1016/j.asoc.2015.06.004>
14. Patel VK, Savsani VJ (2015) Heat transfer search (HTS): a novel optimization algorithm. *Inf Sci* 324:217–246. <https://doi.org/10.1016/j.ins.2015.06.044>
15. Shareef H, Ibrahim AA, Mutlag AH (2015) Lightning search algorithm. *Appl Soft Comput* 36:315–333. <https://doi.org/10.1016/j.asoc.2015.07.028>
16. Wang G-G, Deb S, Cui Z (2015) Monarch butterfly optimization. *Neural Comput Appl* 1–20. <https://doi.org/10.1007/s00521-015-1923-y>
17. Mirjalili S (2016) Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Comput Appl* 27:1053–1073. <https://doi.org/10.1007/s00521-015-1920-1>
18. Aljarah I, Faris H, Mirjalili S (2018) Optimizing connection weights in neural networks using the whale optimization algorithm. *Soft Comput* 22:1–15. <https://doi.org/10.1007/s00500-016-2442-1>
19. K.s. SR, Murugan S (2017) Memory based Hybrid Dragonfly Algorithm for numerical optimization problems. *Expert Syst Appl* 83:63–78. <https://doi.org/10.1016/j.eswa.2017.04.033>
20. Abdel-Basset M, Abdle-Fatah L, Sangaiah AK (2018) An improved Lévy based whale optimization algorithm for bandwidth-efficient virtual machine placement in cloud computing environment. *Clust Comput* 1–16. <https://doi.org/10.1007/s10586-018-1769-z>
21. Hu H, Ao Y, Bai Y, et al (2020) An Improved Harris's Hawks Optimization for SAR Target Recognition and Stock Market Index Prediction. *IEEE Access* 8:65891–65910. <https://doi.org/10.1109/ACCESS.2020.2985596>
22. Ram JP, Babu TS, Dragicevic T, Rajasekar N (2017) A new hybrid bee pollinator flower pollination algorithm for solar PV parameter estimation. *Energy Convers Manag* 135:463–476. <https://doi.org/10.1016/j.enconman.2016.12.082>

23. Zhang X, Xu Y, Yu C, et al (2020) Gaussian mutational chaotic fruit fly-built optimization and feature selection. *Expert Syst Appl* 141:112976. <https://doi.org/10.1016/j.eswa.2019.112976>
24. Gupta D, Sundaram S, Khanna A, et al (2018) Improved diagnosis of Parkinson's disease using optimized crow search algorithm. *Comput Electr Eng* 68:412–424. <https://doi.org/10.1016/j.compeleceng.2018.04.014>
25. Sayed GI, Tharwat A, Hassanien AE (2019) Chaotic dragonfly algorithm: an improved metaheuristic algorithm for feature selection. *Appl Intell* 49:188–205. <https://doi.org/10.1007/s10489-018-1261-8>
26. Wang J, Yang Y, Wang T, et al (2020) Big Data Service Architecture: A Survey. *J Internet Technol* 21:393–405
27. AbdEl-Fattah Sayed S, Nabil E, Badr A (2016) A binary clonal flower pollination algorithm for feature selection. *Pattern Recognit Lett* 77:21–27. <https://doi.org/10.1016/j.patrec.2016.03.014>
28. Ahmed S, Ghosh KK, Singh PK, et al (2020) Hybrid of Harmony Search Algorithm and Ring Theory-Based Evolutionary Algorithm for Feature Selection. *IEEE Access* 8:102629–102645. <https://doi.org/10.1109/ACCESS.2020.2999093>
29. Wang H, Jing X, Niu B (2017) A discrete bacterial algorithm for feature selection in classification of microarray gene expression cancer data. *Knowl-Based Syst* 126:8–19. <https://doi.org/10.1016/j.knosys.2017.04.004>
30. Alazzam H, Sharieh A, Sabri KE (2020) A feature selection algorithm for intrusion detection system based on Pigeon Inspired Optimizer. *Expert Syst Appl* 148:113249. <https://doi.org/10.1016/j.eswa.2020.113249>
31. De Stefano C, Fontanella F, Marrocco C, Scotto di Freca A (2014) A GA-based feature selection approach with an application to handwritten character recognition. *Pattern Recognit Lett* 35:130–141. <https://doi.org/10.1016/j.patrec.2013.01.026>
32. Ma B, Xia Y (2017) A tribe competition-based genetic algorithm for feature selection in pattern classification. *Appl Soft Comput* 58:328–338. <https://doi.org/10.1016/j.asoc.2017.04.042>
33. Lian X, Zhang L, Jiang J, Goss W (2018) An approach for optimized feature selection in large-scale software product lines. *J Syst Softw* 137:636–651. <https://doi.org/10.1016/j.jss.2017.02.044>

34. Xue Y, NanWang, Niu Y, et al (2019) Robust Re-Weighted Multi-View Feature Selection. Comput Mater Contin 60:741–756. <https://doi.org/10.32604/cmc.2019.05611>
35. Erdal Tümer A, Akku,s A (2019) Application of Radial Basis Function Networks with Feature Selection for GDP Per Capita Estimation Based on Academic Parameters. Comput Syst Sci Eng 34:.. <https://www.techscience.com/csse/v34n3/40035>
36. Kanimozhi U, Manjula D (2017) An Intelligent Incremental Filtering Feature Selection and Clustering Algorithm for Effective Classification. Intell Autom Soft Comput 1–9. <https://doi.org/10.1080/10798587.2017.1307626>
37. Fang S, Cai Z, Sun W, et al (2018) Feature Selection Method Based on Class Discriminative Degree for Intelligent Medical Diagnosis. CMC-Comput Mater Contin 55:.. <https://doi.org/10.3970/cmc.2018.02289>
38. Xue Y, Tang T, Pang W, Liu AX (2020) Self-adaptive parameter and strategy based particle swarm optimization for large-scale feature selection problems with multiple classifiers. Appl Soft Comput 88:106031. <https://doi.org/10.1016/j.asoc.2019.106031>
39. Ji B, Lu X, Sun G, et al (2020) Bio-Inspired Feature Selection: An Improved Binary Particle Swarm Optimization Approach. IEEE Access 8:85989–86002. <https://doi.org/10.1109/ACCESS.2020.2992752>
40. Qiu C (2019) A novel multi-swarm particle swarm optimization for feature selection. Genet Program Evolvable Mach. http Journal Pre-proof s://doi.org/10.1007/s10710-019-09358-0
41. Huang C-L, Wang C-J (2006) A GA-based feature selection and parameters optimizationfor support vector machines. Expert Syst Appl 31:231–240. <https://doi.org/10.1016/j.eswa.2005.09.024>
42. Jude Hemanth D, Anitha J (2019) Modified Genetic Algorithm approaches for classification of abnormal Magnetic Resonance Brain tumour images. Appl Soft Comput 75:21–28. <https://doi.org/10.1016/j.asoc.2018.10.054>
43. Krömer P, Platoš J, Nowaková J, Snášel V (2018) Optimal column subset selection for image classification by genetic algorithms. Ann Oper Res 265:205–222. <https://doi.org/10.1007/s10479-016-2331-0>
44. Too J, Abdullah AR (2020) A new and fast rival genetic algorithm for feature selection. J Supercomput 1–31. <https://doi.org/10.1007/s11227-020-03378-9>

45. Wan Y, Wang M, Ye Z, Lai X (2016) A feature selection method based on modified binary coded ant colony optimization algorithm. *Appl Soft Comput* 49:248–258. <https://doi.org/10.1016/j.asoc.2016.08.011>
46. Zhang Y, Liu R, Wang X, et al (2020) Boosted binary Harris hawks optimizer and feature selection. *Eng Comput*. <https://doi.org/10.1007/s00366-020-01028-5>
47. Too J, Abdullah AR (2020) Chaotic Atom Search Optimization for Feature Selection. *Arab J Sci Eng*. <https://doi.org/10.1007/s13369-020-04486-7>
48. Kanti Ghosh K, Guha R, Ghosh S, et al (2020) Atom Search Optimization with Simulated Annealing -- a Hybrid Metaheuristic Approach for Feature Selection. *ArXiv E-Prints 2005:arXiv:2005.08642*
49. Too J, Abdullah AR (2020) Binary atom search optimisation approaches for feature selection. *Connect Sci* 0:1–25. <https://doi.org/10.1080/09540091.2020.1741515>
50. Baynath P, Soyjaudah S, Khan M (2018) Feature Selection and Representation of Evolutionary Algorithm on Keystroke Dynamics. *Intell Autom Soft Comput* 1--1. <https://doi.org/10.31209/2018.100000060>
51. Mirjalili S, Mirjalili SM, Lewis A (2014) Grey Wolf Optimizer. *Adv Eng Softw* 69:46– 61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
52. Emary E, Zawbaa HM, Hassanien AE (2016) Binary grey wolf optimization approaches for feature selection. *Neurocomputing* 172:371–381. <https://doi.org/10.1016/j.neucom.2015.06.083>
53. Zawbaa HM, Emary E, Grosan C, Snasel V (2018) Large-dimensionality small-instance set feature selection: A hybrid bio-inspired heuristic approach. *Swarm Evol Comput* 42:29–42. <https://doi.org/10.1016/j.swevo.2018.02.021>
54. Chantar H, Mafarja M, Alsawalqah H, et al (2019) Feature selection using binary grey wolf optimizer with elite-based crossover for Arabic text classification. *Neural Comput Appl*. <https://doi.org/10.1007/s00521-019-04368-6>
55. Too J, Abdullah AR (2020) Opposition based competitive grey wolf optimizer for EMG feature selection. *Evol Intell*. <https://doi.org/10.1007/s12065-020-00441-5>
56. Abdel-Basset M, El-Shahat D, El-henawy I, et al (2020) A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection. *Expert Syst Appl* 139:112824. <https://doi.org/10.1016/j.eswa.2019.112824>

57. Mafarja M, Mirjalili S (2018) Whale optimization approaches for wrapper feature selection. *Appl Soft Comput* 62:441–453. <https://doi.org/10.1016/j.asoc.2017.11.006>
58. Mafarja MM, Mirjalili S (2017) Hybrid Whale Optimization Algorithm with simulated annealing for feature selection. *Neurocomputing* 260:302–312. <https://doi.org/10.1016/j.neucom.2017.04.053>
59. Tubishat M, Abushariah MAM, Idris N, Aljarah I (2019) Improved whale optimization algorithm for feature selection in Arabic sentiment analysis. *Appl Intell* 49:1688–1707. <https://doi.org/10.1007/s10489-018-1334-8>
60. Shaban WM, Rabie AH, Saleh AI, Abo-Elsoud MA (2020) A new COVID-19 Patients Detection Strategy (CPDS) based on hybrid feature selection and enhanced KNN classifier. *Knowl-Based Syst* 205:106270. <https://doi.org/10.1016/j.knosys.2020.106270>
61. Jain G, Mittal D, Thakur D, Mittal MK (2020) A deep learning approach to detect Covid19 coronavirus with X-Ray images. *Biocybern Biomed Eng* 40:1391–1405. <https://doi.org/10.1016/j.bbe.2020.08.008>
62. Tuncer T, Dogan S, Ozyurt F (2020) An automated Residual Exemplar Local Binary Pattern and iterative ReliefF based COVID-19 detection method using chest X-ray image. *Chemom Intell Lab Syst* 203:104054. <https://doi.org/10.1016/j.chemolab.2020.104054>
63. Ribeiro MHDM, da Silva RG, Mariani VC, Coelho L dos S (2020) Short-term forecasting COVID-19 cumulative confirmed cases: Perspectives for Brazil. *Chaos Solitons Fractals* 135:109853. <https://doi.org/10.1016/j.chaos.2020.109853>
64. da Silva RG, Ribeiro MHDM, Mariani VC, Coelho L dos S (2020) Forecasting Brazilian and American COVID-19 cases based on artificial intelligence coupled with climatic exogenous variables. *Chaos Solitons Fractals* 139:110027. <https://doi.org/10.1016/j.chaos.2020.110027>
65. Mafarja MM, Eleyan D, Jaber I, et al (2017) Binary Dragonfly Algorithm for Feature Selection. In: 2017 International Conference on New Trends in Computing Sciences (ICTCS). pp 12–17
66. Reynolds CW (1987) Flocks, herds and schools: a distributed behavioral model. *ACM SIGGRAPH Comput Gr* 21:25–34
67. Yang X-S (2010) Nature-inspired metaheuristic algorithms, 2nd edn. Luniver Press

68. Cui Z, Shi Z (2009) Boid particle swarm optimisation. *Int J Innov Comput Appl* 2:77–85
69. Kadrovach BA, Lamont GB (2002) A particle swarm model for swarm-based networked sensor systems. In: *Proceedings of the 2002 ACM symposium on applied computing*, pp 918–924
70. Cui Z (2009) Alignment particle swarm optimization. In: *Cognitive informatics, 2009. ICCI'09. 8th IEEE international conference on*, pp 497–501
71. Mirjalili S, Lewis A (2013) S-shaped versus V-shaped transfer functions for binary particle swarm optimization. *Swarm Evolut Comput* 9:1–14
72. Saremi S, Mirjalili S, Lewis A (2014) How important is a transfer function in discrete heuristic algorithms. *Neural Comput Appl*:1–16
73. Mirjalili S, Wang G-G, Coelho LDS (2014) Binary optimization using hybrid particle swarm optimization and gravitational search algorithm. *Neural Comput Appl* 25:1423–1435
74. Sayed GI, Tharwat A, Hassanien AE (2019) Chaotic dragonfly algorithm: an improved metaheuristic algorithm for feature selection. *Appl Intell* 49:188–205. <https://doi.org/10.1007/s10489-018-1261-8>
75. Emary E, Zawbaa HM, Hassanien AE (2016) Binary ant lion approaches for feature selection. *Neurocomputing* 213:54–65. <https://doi.org/10.1016/j.neucom.2016.03.101>
76. Mafarja MM, Eleyan D, Jaber I, et al (2017) Binary Dragonfly Algorithm for Feature Selection. In: *2017 International Conference on New Trends in Computing Sciences (ICTCS)*. pp 12–17
77. Mafarja, M. M., Eleyan, D., Jaber, I., Hammouri, A., & Mirjalili, S. (2017). *Binary Dragonfly Algorithm for Feature Selection. 2017 International Conference on New Trends in Computing Sciences (ICTCS)*. doi:10.1109/ictcs.2017.43
78. Al-Tashi, Q., Abdulkadir, S. J., Rais, H. M., Mirjalili, S., & Alhussian, H. (2019). *Binary Optimization Using Hybrid Grey Wolf Optimization for Feature Selection. IEEE Access, 1–1*. doi:10.1109/access.2019.2906757
79. Chantar, H., Tubishat, M., Essgaer, M., & Mirjalili, S. (2021). *Hybrid Binary Dragonfly Algorithm with Simulated Annealing for Feature Selection. SN Computer Science, 2(4)*. doi:10.1007/s42979-021-00687-5

